

```

import torch
import numpy as np
import torch.nn as nn
from FlowMatching import Flow
from tqdm import tqdm
import torch.optim as optim
import matplotlib
import matplotlib.pyplot as plt
import pandas as pd

flow = Flow()
mixture_prob = np.array([0.55, 0.45], dtype=float)
mixture_mus = np.array([-0.85, 1.5], dtype=float)
mixture_sigmas = np.array([0.65, 0.25], dtype=float)

def interp(x0, x1, t):
    xt = ((1-t) * x0) + (t * x1)
    return xt
def targVel(x0, x1):
    return x1 - x0
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def computeLoss(model, x1, condition):
    batch_size = x1.shape[0]
    t = torch.rand(batch_size, 1)
    x0 = torch.randn_like(x1)
    xt = interp(x0, x1, t)
    vTarg = targVel(x0, x1)
    vPred = model(t, xt, condition)
    loss = torch.mean((vPred - vTarg) ** 2)
    return loss

def generateTraining(batch_size = 64, p_condition = 0.55):
    conditions = np.random.choice([0, 1], size=batch_size, p =
[p_condition, 1.0-p_condition])
    means = np.where(conditions == 0, -0.85, 1.5)
    stds = np.where(conditions == 0, 0.65, 0.25)
    x1 = np.random.normal(loc=means, scale=stds)
    return conditions, x1

def mixture_sample(condition):
    if condition == 0:
        return np.random.normal(loc=-0.85, scale = 0.65)
    elif condition == 1:
        return np.random.normal(loc=1.5, scale = 0.25)
    else:
        raise ValueError("Condition must be 0 or 1.")

```

```

def display_figure(fig, *, altText):
    display(fig, metadata = {"alt": altText})
    plt.close(fig)

class FlowMatchingModel(nn.Module):
    def __init__(self, data_dim, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(data_dim + 1, hidden_dim),
nn.GELU(), nn.Linear(hidden_dim, hidden_dim), nn.GELU(),
nn.Linear(hidden_dim, data_dim),)

    def forward(self, t, x_t, condition):
        txc = torch.cat([t, x_t, condition], dim=-1)
        return self.net(txc)

dDim = 2
hiddenDim = 64
trainIters = 10000
lr = 1e-3
batchSize = 256
np.random.seed(42)
torch.manual_seed(626)
model = FlowMatchingModel(dDim, hiddenDim).to(DEVICE).train()
optimizer = optim.Adam(model.parameters(), lr)
losses = []
with tqdm(range(trainIters), desc="Training", unit="iteration") as
progress_bar:
    for i in progress_bar:
        x1 =
torch.from_numpy(mixture_sample(batchSize)).to(dtype=torch.float32,
device=DEVICE).unsqueeze(-1)
        x0 = torch.randn_like(x1)
        t = torch.rand(x1.shape[0], device=DEVICE).unsqueeze(-1)
        loss = computeLoss(model, x0, x1, t)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
        progress_bar.set_postfix({"Loss": f"{loss.item():.2f}"})

Training: 100%|██████████| 10000/10000 [00:15<00:00,
629.37iteration/s, Loss=1.90]

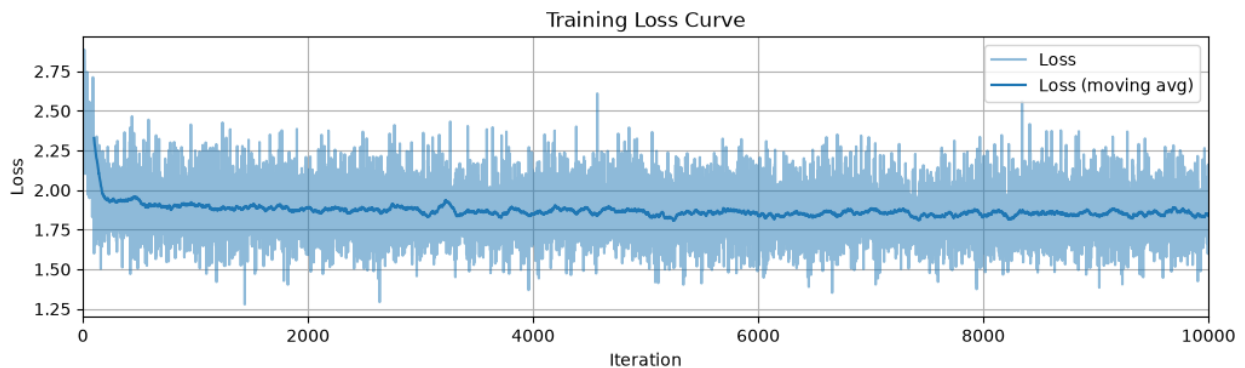
fig, ax = plt.subplots(figsize=(12, 3), dpi=100)
ax.plot(losses, color="tab:blue", alpha=0.5, label="Loss")
ax.set_xlabel("Iteration")
ax.set_ylabel("Loss")
ax.set_title("Training Loss Curve")

```

```

# Plot a smoothed loss curve using a simple moving average
window_size = 100
smoothed_losses = np.convolve(losses, np.ones(window_size) /
window_size, mode="valid")
ax.plot(np.arange(window_size - 1, len(losses)), smoothed_losses,
color="tab:blue", label="Loss (moving avg)")
ax.legend(loc="upper right")
ax.set_xlim(0, len(losses))
ax.grid(True)
display_figure(
    fig,
    altText="Training loss over optimization steps.",
)
del fig, ax, window_size, smoothed_losses

```



```

fig, (ax1, ax2, ax3) = plt.subplots(
    1,
    3,
    figsize=(12, 5),
    gridspec_kw={"width_ratios": [1, 5, 1]},
    sharey=True,
    dpi=100,
    constrained_layout=True,
)
x_min, x_max = -2.2, 2.2 # Narrow view for flow field image
x_min_wide, x_max_wide = -3.2, 3.2 # Wide view for sample paths

# Sample set of noise and target points
data_size: int = 100000
np.random.seed(1) # Set random seeds for reproducibility
data_x_0 = np.random.randn(data_size)

# Plot the velocity field #####
# Plot Noise samples X0
ax1.set_title(r"Noise Samples $X_0$")
ax1.hist(

```

```

        data_x_0.flatten(),
        bins=100,
        alpha=0.5,
        label=r"Noise Samples $X_0$",
        color="tab:orange",
        density=True,
        orientation="horizontal",
    )
ax1.invert_xaxis()
ax1.set_ylabel("x")
ax1.set_xlabel("density")

# Plot Flow Field
n_flow_steps = 100
# Set up the flow field histogram parameters
img_hist_size = 200
# Narrow view for flow field image
flow_field_img_bins = np.zeros((n_flow_steps + 1, img_hist_size))
flow_field_img_x_bin_edges = np.linspace(x_min, x_max, img_hist_size + 1)
flow_field_img_x_bin_centers = (
    torch.from_numpy((flow_field_img_x_bin_edges[:-1] +
flow_field_img_x_bin_edges[1:]) / 2)
    .float()
    .to(DEVICE)
    .unsqueeze(-1)
)

# Wide view to compute the pathlines, need to be wide enough because
paths can flow out of the narrow view
flow_field_for_path_bins = np.zeros((n_flow_steps + 1, img_hist_size))
flow_field_for_path_x_bin_edges = np.linspace(x_min_wide, x_max_wide,
img_hist_size + 1)
flow_field_for_path_x_bin_centers = (
    torch.from_numpy((flow_field_for_path_x_bin_edges[:-1] +
flow_field_for_path_x_bin_edges[1:]) / 2)
    .float()
    .to(DEVICE)
    .unsqueeze(-1)
)

# Build up the histogram of the reference paths by going over the
discretized t-bins
# We're building 2 histograms:
# - `flow_field_img_bins` narrow view for the flow field image, might
have NaNs if there are no samples in a bin
# - `flow_field_for_path_bins` wide view for the pathlines, avoid NaNs
by setting the velocity to 0 if there are no samples in a bin
with torch.inference_mode():

```

```

    model.eval()
    for i, t in enumerate(torch.linspace(0, 1, n_flow_steps + 1)):
        t = t.expand_as(flow_field_img_x_bin_centers).to(DEVICE)
        # Get the model's prediction for the velocity field at the bin
        centers
        flow_field_img_bins[i] = model(t=t,
x_t=flow_field_img_x_bin_centers).cpu().numpy().squeeze()
        flow_field_for_path_bins[i] = (
            model(t=t,
x_t=flow_field_for_path_x_bin_centers).cpu().numpy().squeeze()
        )

# Plot the flow field
max_abs_flow_field = np.nanmax(np.abs(flow_field_img_bins))
color_norm = matplotlib.colors.TwoSlopeNorm(vmin=-max_abs_flow_field,
vcenter=0.0, vmax=max_abs_flow_field)
im = ax2.imshow(
    flow_field_img_bins.T,
    aspect="auto",
    origin="lower",
    extent=[0, 1, x_min, x_max],
    cmap="coolwarm",
    norm=color_norm,
)
ax2.set_ylim(x_min, x_max)
cbar = fig.colorbar(
    im,
    ax=[ax1, ax2, ax3],
    orientation="horizontal",
    fraction=0.08,
    aspect=40,
    pad=0.04,
)
cbar.set_label("Velocity field value (red pushes up, blue pulls
down)")

# Create pathlines from the flow field
n_paths = 9
# Initialize tensor to store the flow process
paths = np.zeros((n_flow_steps + 1, n_paths))
x_t = torch.linspace(start=x_min_wide, end=x_max_wide, steps=n_paths,
device=DEVICE).unsqueeze(-1)
paths[0] = x_t.cpu().squeeze().numpy()
# Generate the flow process
with torch.inference_mode():
    model.eval()
    ts = torch.linspace(0, 1, n_flow_steps + 1)
    for i in range(n_flow_steps):
        t = ts[i].expand_as(x_t).to(DEVICE)

```

```

    dt = ts[i + 1] - ts[i]
    x_t = x_t + model(t=t, x_t=x_t.to(DEVICE)) * dt
    paths[i + 1] = x_t.cpu().numpy().squeeze()

# Plot the pathlines with arrows using quiver, following best
practices
t_coords = np.linspace(0, 1, n_flow_steps + 1)
arrow_stride = 35 # space out arrows for clarity
arrow_offset = 7 # offset the arrows to the right for clarity
for i in range(n_paths):
    y = paths[:, i]
    idx = np.arange(arrow_offset, len(t_coords) - 1, arrow_stride)
    x0 = t_coords[idx]
    y0 = y[idx]
    u = np.diff(t_coords)[idx]
    v = np.diff(y)[idx]
    ax2.quiver(
        x0,
        y0,
        u,
        v,
        angles="xy",
        scale_units="xy",
        scale=0.6,
        units="inches",
        width=0.015,
        headwidth=6,
        headlength=9,
        headaxislength=7,
        pivot="tail",
        color="dimgray",
        alpha=0.9,
        zorder=2,
    )
# overlay original line on top of arrows for clarity
ax2.plot(
    t_coords,
    y,
    linestyle="-",
    color="dimgray",
    linewidth=1.5,
    alpha=0.7,
    zorder=3,
    label=f"Sample Path {i + 1}",
)

ax2.set_xlabel("$t$: flow step")
ax2.set_title(r"Predicted velocity field  $\{FM\}_{\theta}(x_t, t)$  with
pathlines")

```

```

ax2.grid(False)

# Plot target data distribution x1

# Sample target data distribution x1
x_t = torch.from_numpy(data_x_0).float().to(DEVICE).unsqueeze(-1)
dt: float = 0.01 # Step size for Euler integration
n_flow_steps = int(1 / dt)
# Generate the flow process
with torch.inference_mode():
    for i in range(n_flow_steps):
        t = torch.full_like(x_t, i * dt, device=DEVICE)
        x_t = x_t + model(t=t, x_t=x_t.to(DEVICE)) * dt
data_x_1 = x_t.cpu().numpy().squeeze()

ax3.set_title(r"Predicted Data  $\hat{X}_1$ ")
ax3.hist(
    data_x_1.flatten(),
    bins=100,
    alpha=0.5,
    label=r"Predicted Data  $\hat{X}_1$ ",
    color="tab:blue",
    density=True,
    orientation="horizontal",
)
ax3.set_xlabel("density")
ax3.yaxis.set_label_position("right")
ax3.set_ylabel("x")
# Also show y-axis values on the right side
ax3.yaxis.set_visible(True)
ax3.yaxis.set_tick_params(labelright=True)

display_figure(
    fig,
    altText="Learned velocity field, generated paths, and
reconstructed output distribution.",
)
del (
    fig,
    ax1,
    ax2,
    ax3,
    x_min,
    x_max,
    x_min_wide,
    x_max_wide,
    data_size, # fmt: skip
    data_x_0,
    data_x_1,

```

```

dt,
n_flow_steps,
img_hist_size,
flow_field_img_bins, # fmt: skip
flow_field_img_x_bin_edges,
flow_field_for_path_bins,
flow_field_for_path_x_bin_edges, # fmt: skip
y,
idx,
x0,
y0,
u,
i,
t,
x_t, # fmt: skip
)

```

